

Ontology. 지식 그래프, GraphRAG

데이터 저장 중심에서 의미 모델 중심으로 이동

1. 문제 정의

- 대규모 데이터 환경에서 semantic mismatch(의미상 불일치) 발생
- LLM 응답의 사실성 및 검증 가능성 한계
- 벡터 유사도 중심 검색의 제약 조건 처리 한계
- 결과적으로 “의미 모델 + 규칙 + 검증” 결합 구조 필요.

2. Ontology의 정의

“Gruber(1993) : An explicit specification of a conceptualization ”

- **Conceptualization : 도메인에 대한 추상적 개념화**
- **Explicit specification : 개념/관계/제약의 명시적 기술**
- **Formal semantics : 기계 추론 가능한 형식 논리 기반 표현**

2-1. FOL(First-Order Logic)

First-Order Logic(FOL) : 복잡한 지식을 논리적으로 표현하는 언어

1. 데이터에 의미와 맥락 부여
2. 규칙 기반 자동 추론 가능
3. 복잡한 질의와 검색 지원
4. 데이터 통합과 일관성 확보



2-2. 온톨로지와 FOL 차이?

온톨로지와 FOL은 상호보완적인 관계이지만 명확한 역할 구분

- 온톨로지 : 무엇이 존재하는지? 관계를 정의
- FOL : 무엇이 사실인가? 사실과 규칙을 논리적으로 표현

	온톨로지	FOL
역할	의미 구조 정의 (Concepts, Relations)	의미 있는 사실 표현 (Facts, Rules)
예시	Class: Station, Property: hasPopulation	"Population(Gangnam, 92000)" " $\forall x$ (HasCongestionLevel(x, 붐빔) $\rightarrow$ VeryCrowded(x))"
목적	데이터의 개념화 (Conceptualization)	추론 가능한 명제화 (Formalization)
구현 언어	RDF, OWL, RDFS	Prolog, 술어 논리식
질문 관점	무엇이 존재하는가? (What exists?)	무엇이 사실인가? (What is true?)

2-3. Ontology 구성요소



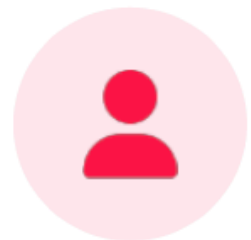
클래스

Class

개념이나 사물의 종류를 정의하는
추상적 집합

가족 관계도 예시

'사람', '남자', '여자'와 같은 개념
종류



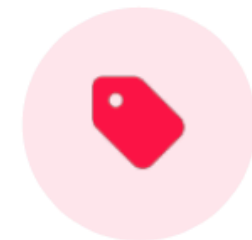
인스턴스

Instance

클래스에 속하는 실제적인 개체

가족 관계도 예시

'홍길동', '성춘향'과 같은 실제 인
물



속성

Property

인스턴스가 가지는 특징이나 값을
표현

가족 관계도 예시

'홍길동'의 '나이: 25세', '직업: 학
생'



관계

Relation

인스턴스나 클래스 간의 연결과
상호작용

가족 관계도 예시

'홍길동'은 '성춘향'과 '친구이다
(isFriendOf)'



공리

Axiom

반드시 참이 되는 규칙이나 제약
조건

가족 관계도 예시

'모든 남자는 사람이다', '부모의 부
모는 조부모이다'

2-4. TBox 와 ABox : 지식의 두 기둥 (지식 베이스)



- TBox(Terminological Box) : 용어/개념/관계 정의
- ABox(Assertional Box) : 개체 사실(assertion) 정의
- 장점 : 모델 재사용성, 데이터 확장성, 유지보수성 강화

2-5. TBox : 지식의 설계도



세상의 문법책

또는



건축 설계도



Classes (클래스)

Place: 장소 개념

Forecast: 예측 개념

CongestionLevel: 혼잡도 수준 개념

SubClassOf: 계층 관계 표현



Properties (속성)

hasForecast: Object Property

hasCongestionLevel: Object Property

avgPopulation: Data Property

객체 간 관계 및 데이터 값 표현



Constraints (제약조건)

Domain/Range: 속성의 적용 범위

Disjointness: 배타적 관계

Cardinality: 관계의 수량적 제약

데이터 정합성을 보장하는 규칙들



Best Practices

- 명명 규칙과 URI 일관성 유지
- Domain/Range 명확히 설정하고 필요시 Disjoint 관계 정의
- OWL의 개방 세계 가정(Open World Assumption) 고려
- 필요한 경우 폐쇄 세계 제약을 명시적으로 모델링



Protégé에서의 TBox 설계

- Classes 탭: 클래스 계층 구조 설계
- Object Properties 탭: 객체 간 관계 정의
- Data Properties 탭: 데이터 값 속성 정의
- Reasoner: 논리적 일관성 검증

- TBox : 무엇을 말할 수 있는가의 틀

2-6. ABox : 현실 세계의 기록



Instances (Individuals)

TBox에서 정의한 클래스의 실제 사례로, 세계에 존재하는 구체적인 개체들

```
gangnam_station rdf:type Place
forecast_1500 rdf:type Forecast
level_crowded rdf:type CongestionLevel
```



Property Assertions

인스턴스들 간의 관계와 데이터 값을 기술하는 실제 사실 진술

```
gangnam_station hasForecast forecast_1500
forecast_1500 hasCongestionLevel level_crowded
forecast_1500 avgPopulation 92000
```



ABox 설계 모범 사례 (Best Practices)

- ▶ 인스턴스에 명확한 라벨과 주석 부여
(rdfs:label, rdfs:comment)
- ▶ 동일 실체 정합성 관리
(owl:sameAs 적절한 사용)
- ▶ 시간 관련 메타데이터 포함
(생성일, 수정일, 버전 등)
- ▶ 데이터 품질 검증
(SHACL, OWL 제약 등 활용)

- ABox : 문법에 맞춰 쓰인 구체적인 문장들/설계도로 지은 실제 건물

2-7. Object Property Assertions (관계연결)

관계 설정 - Object Property



실제 인스턴스 관계

```
:forecast_POI014_20250926T160000 :forPlace  
:place_POI014 .
```

Domain / Range 이해하기

- Domain: forPlace의 주어 타입 = Forecast
- Range: forPlace의 목적어 타입 = Place
- 관계 방향: Forecast가 Place를 참조하는 방향
- 위반 사례: Place에서 Forecast를 가리키면 안 됨

Protégé 경로 안내

- 1 Individuals 탭 열기 > Individuals by class
- 2 Forecast 클래스 선택 > forecast_POI014_20250926T160000 인스턴스 클릭
- 3 Object property assertions 섹션 찾기
- 4 + 버튼 클릭 > forPlace 프로퍼티 선택
- 5 place_POI014 인스턴스 선택 > OK

체크리스트

- ✓ 방향 확인: forPlace의 Domain=Forecast, Range=Place
- ✓ 중복 연결 방지 (동일 인스턴스에 다중 값 검토)
- ✓ Response.aboutPlace (응답→장소)도 동일 패턴 적용 가능

- Object Property Assertions는 두 인스턴스 간의 관계를 정의하여 지식그래프의 연결을 만듦

2-8. Data Property Assertions

Forecast 데이터 프로퍼티 입력

Protégé 단계

```
/* Individuals by class 탭에서 */
1. Forecast 인스턴스 선택 ( forecast_POI014_20250926T160000 )
2. Data property assertions 영역 확인
3. +버튼 클릭 → 프로퍼티 선택 창
4. 원하는 데이터 프로퍼티 선택 (예: forecastCongestionLevel)
5. 값 입력 및 타입 지정
/* 필요시 언어태그 지정 (한글 텍스트) */
Language Tag :@ko
```

필수 데이터 프로퍼티:

- forecastCongestionLevel *xsd:string*
- forecastPopulationMax *xsd:integer*
- forecastPopulationMin *xsd:integer*
- forecastTime *xsd:dateTime*

Turtle 표현 (TTL 형식)

데이터 프로퍼티 입력 결과 (TTL)

```
:forecast_POI014_20250926T160000
:forecastCongestionLevel "붐뵠" ;
:forecastPopulationMax 96000 ;
:forecastPopulationMin 94000 ;
:forecastTime "2025-09-26T16:00:00" ^^xsd:dateTime ;
:forPlace :place_POI014 .
```

데이터 타입 지정 방법

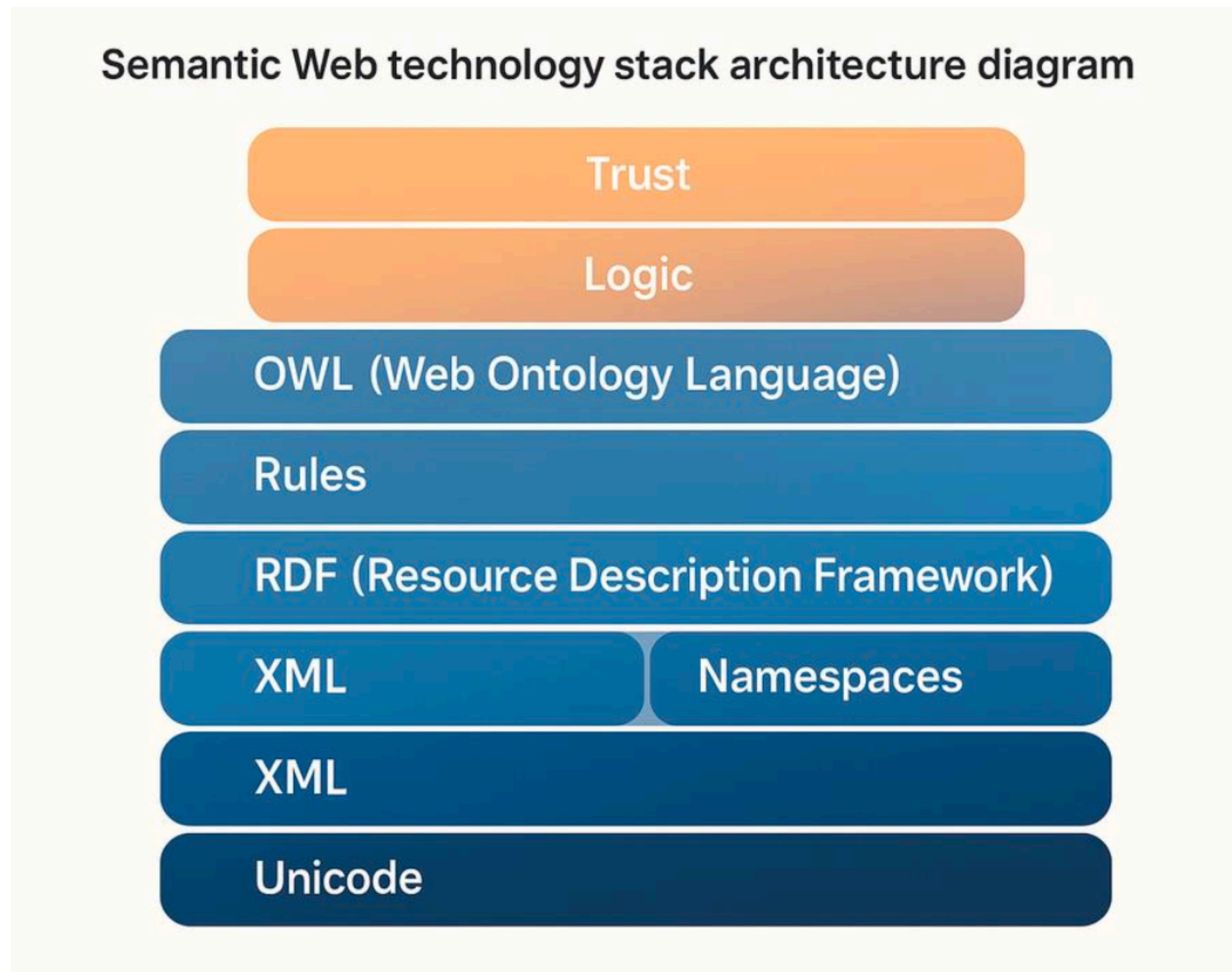
문자열(String): 기본 타입, 별도 지정 없이 큰따옴표 안에 입력

정수(Integer): 타입 드롭다운에서 "xsd:integer" 선택

날짜/시간: ISO 형식("YYYY-MM-DDThh:mm:ss")으로 입력 후 "xsd:dateTime" 선택

불리언(Boolean): "true"/"false" 입력, "xsd:boolean" 선택

3. 시맨틱 웹 표준 스택



- URI/IRI: 자원 식별
- RDF: 트리플 구조 기반 사실 표현
- RDFS: 기본 스키마/분류 체계
- OWL: 고급 논리 제약 및 추론
- SPARQL: 그래프용 SQL
- SHACL: 제약 기반 데이터 검증

3-1. 시맨틱 웹 표준 스택

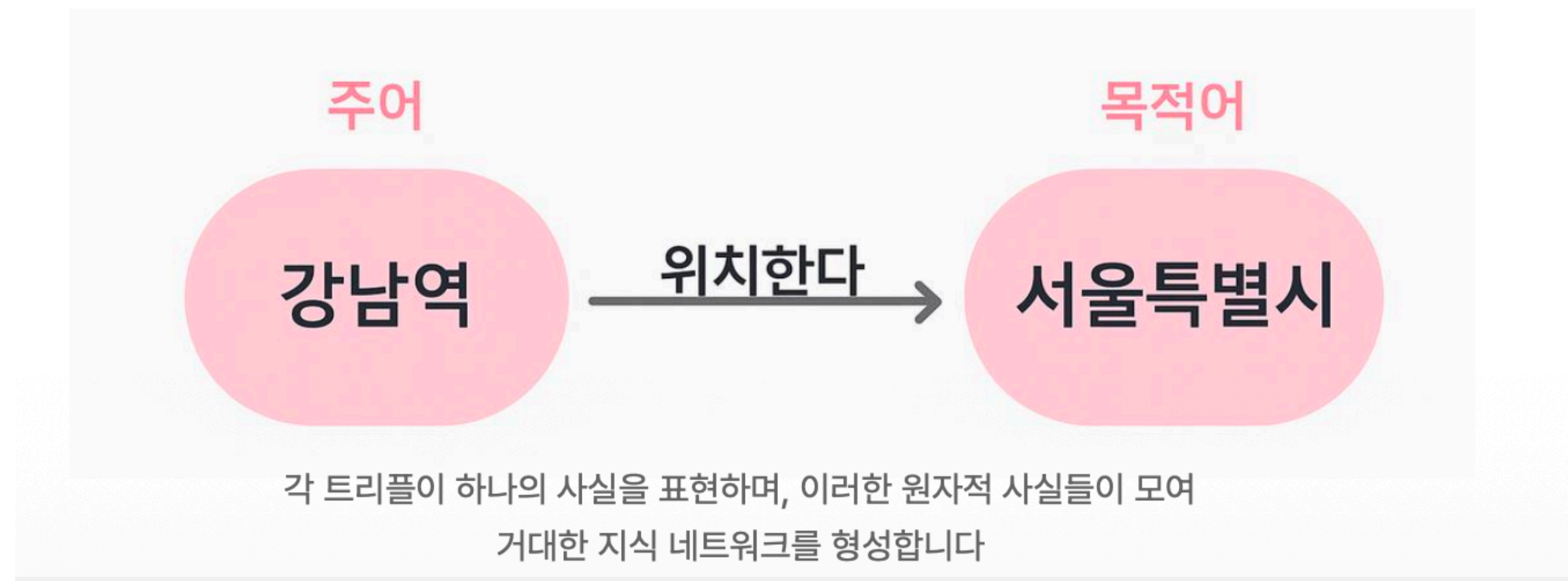
- 시맨틱 웹의 비전

데이터에 의미(semantic)를 부여하여 기계가 정보를 이해하고 추론할 수 있게 하는 패러다임

1. 기계가 데이터의 맥락을 이해
2. 서로 다른 데이터셋 간 통합 용이
3. 질의와 검색의 정확도 향상
4. 자동화된 추론과 의사결정 지원

3-2. RDF : 지식의 ATOM

- ✓ Subject-Predicate-Object: 주어(자원), 서술어(관계/속성), 목적어(값/자원)로 모든 지식 표현
- ✓ 그래프 데이터 모델: 노드와 엣지로 구성된 네트워크 형태로 지식 연결
- ✓ URI 기반 식별: 모든 자원과 관계를 고유 URI로 명확히 식별하여 전역 참조
- ✓ 스키마-프리 확장성: 새로운 정보를 언제든지 추가 가능한 유연한 구조
- ✓ 웹 스케일 연결: 수십억 개의 사실을 서로 연결하여 거대한 지식 그래프 구성



• RDF의 예

잠실롯데월드 - 직원 수는 - 10만명이다

3-3. OWL : 지식의 문법

역할 (Role)



분류 체계 지원

클래스 계층 구조와 상속 관계를 명확하게 정의



제약 조건 표현

도메인/범위 제약, 속성 제약, 논리적 제약 명시



추론 규칙 지원

명시적 지식에서 암묵적 지식을 도출하는 논리적 규칙

논리적 표현력

효과 (Effect)



일관성 검사

데이터가 정의된 제약과 충돌하지 않는지 자동 확인



자동 분류

인스턴스를 속성에 기반해 적절한 클래스로 자동 배치



숨겨진 관계 발견

직접 명시되지 않은 관계를 추론 엔진이 도출

지능형 데이터 활용

3-3. OWL : 지식의 문법

간단한 OWL 예시



🎓 클래스 계층 관계

클래스 간 계층 구조 정의:

```
Student ⊑ Person  
Professor ⊑ Person
```

⚖️ 동등 관계와 논리적 조합

클래스 의미를 논리적으로 정의:

```
Father ≡ Male ∧ (hasChild some Person)
```

형식적 의미론

표현력과 활용



🏗️ OWL 관계 유형

주요 논리적 관계:

- > 하위 클래스: 포함 관계
- > 동등 클래스: 의미적 동일성
- > 상호 배타: 겹치지 않는 개념

⚙️ 추론 기반 장점

명시적 규칙으로 암묵적 지식 도출:

- > 일관성 검사: 모순 탐지
- > 숨은 관계 발견: 암묵적 연결

3-4. SPARQL : 지식을 향한 질문

개념 (Concept)



RDF 그래프 패턴 매칭

SPARQL은 그래프 데이터를 대상으로 특정 패턴과 일치하는 부분을 찾는 방식으로 동작합니다. SQL이 테이블을 대상으로 한다면, SPARQL은 그래프를 대상으로 합니다.

- ✓ 주어-서술어-목적어(S-P-O) 패턴을 기반으로 질의
- ✓ 변수(?x)를 사용해 원하는 정보 추출
- ✓ RDF 트리플 스토어에 저장된 데이터 검색

특징 (Features)



의미 기반 탐색

키워드 일치가 아닌 의미 관계를 기반으로 연관된 정보를 찾을 수 있어 복잡한 질문에 대한 답변이 가능합니다.

- ✓ 페더레이션(FEDERATED) 쿼리 지원
- ✓ 다양한 엔드포인트의 데이터를 통합 검색
- ✓ OWL 온톨로지와 연계하여 추론 질의 가능

시맨틱 웹의 질의 언어

3-4. SPARQL : 지식을 향한 질문

SPARQL 쿼리 예시



🔍 서울 내 혼잡도 검색 예시

다음 SPARQL 쿼리는 서울에 위치하면서 혼잡도가 "붐빔" 상태인 모든 장소를 검색합니다.

```
PREFIX ex: <http://example.org/>
SELECT ?place WHERE {
  ?place ex:locatedIn ex:Seoul ;
  ex:congestion "붐빔" .
}
```

패턴 기반 매칭

복합 조건의 간결한 표현



💡 SQL과의 차이점

SPARQL은 의미 관계를 기반으로 데이터를 탐색하여 복잡한 조건을 보다 자연스럽게 표현할 수 있습니다.

- ✓ 세미콜론(;)으로 같은 주어에 대한 여러 조건을 간결하게 나열
- ✓ 그래프 패턴으로 여러 단계의 관계 추적 가능
- ✓ OPTIONAL, FILTER 등으로 조건부 매칭

3-5. 온톨로지와 시맨틱 웹

핵심 요소의 역할



RDF = 뼈대

지식 체계의 기본 구조를 이루는 트리플 형태의 기초 프레임워크



OWL = 두뇌

논리적 규칙과 제약으로 추론과 일관성 검사를 가능하게 하는 지능



SPARQL = 신경계

정보를 질의하고 응답을 전달하는 시스템 내 통신 체계



URI/Linked Data = 혈관

필수 자원을 식별하고 시스템 전체에 연결하는 순환 체계

통합된 지식 생태계



핵심 메시지

인체의 여러 부위가 서로 유기적으로 연결되어 생명 시스템을 이루듯이, 온톨로지와 시맨틱 웹의 각 구성요소도 독립적으로는 제한적이지만 함께 작동할 때 지능적인 지식 시스템을 형성합니다.

온톨로지가 도메인 지식의 의미론적 설계도라면, 시맨틱 웹은 이 설계도를 웹 규모로 실행하는 기술 인프라입니다.

66

"시맨틱 웹은 인간의 두뇌가 연결되는 방식처럼, 컴퓨터가 의미를 연결할 수 있게 합니다."

99

4. 온톨로지와 RDBMS 구조적 차이

RDBMS



기본 단위

테이블의 행(row) - 미리 정의된 스키마에 맞춰 저장된 레코드

관계 표현

외래 키(FK) - 다른 테이블을 가리키는 숫자 ID로 간접 연결

스키마 변화

고정적 - 구조 변경이 어려우며 큰 비용 발생 (ALTER TABLE)

의미 수준

데이터 수준 - 값 중심의 저장과 구조적 표현에 초점

질의 언어

SQL - 구조화된 테이블 대상 질의

저장과 관리 중심

온톨로지



기본 단위

트리플(Triple) - 주어-서술어-목적어 형태의 지식 원자

관계 표현

명시적 관계(URI) - 의미를 가진 이름으로 직접 연결

스키마 변화

유연함 - 레고 블록처럼 새 개념과 관계를 쉽게 추가 가능

의미 수준

개념과 논리 수준 - 의미와 맥락 표현에 초점

질의 언어

SPARQL - 그래프 패턴 기반 질의

의미 연결과 추론 중심

4-1. RDBMS의 한계와 온톨로지의 역할

RDBMS의 한계

데이터베이스의 구조적 제약

RDBMS는 효율적인 저장과 관리에 최적화되어 있지만, 데이터의 의미와 맥락을 표현하고 확장하는 데는 한계가 있습니다.

데이터 사일로 문제

각 시스템마다 다른 스키마와 구조로 인해 통합이 어려움

추론 능력 부재

명시적으로 저장된 데이터만 조회 가능, 암묵적 관계 도출 불가

표준화 어려움

시스템마다 상이한 데이터 모델로 재사용과 호환성 문제

스키마 변경 비용

새로운 데이터 유형이나 관계 추가 시 구조적 변경 필요

온톨로지의 보완 영역

의미적 관계와 지식 확장

온톨로지는 RDBMS가 가지지 못한 네 가지 중요한 영역을 보완하여 데이터에서 지식으로의 확장을 가능하게 합니다.

1. 데이터 통합 (Integration)

서로 다른 데이터베이스 스키마도 공통 개념(URI)으로 의미적 통합, 데이터 사일로 허물기

2. 추론 (Reasoning)

논리 규칙을 통해 명시적으로 저장되지 않은 새로운 지식을 자동 도출

3. 지식 재사용 (Reuse)

표준 온톨로지를 공유 어휘로 활용하여 데이터의 일관성과 연속성 확보

4. 확장성 (Extensibility)

테이블 구조 변경 없이 레고 블록처럼 새로운 개념과 관계를 유연하게 추가

의미적 데이터 활용

4-2. 데이터 사례 비교

RDBMS 저장 구조

개별 테이블 중심 구조

인구, 상권, 교통 데이터가 각각 별도의 테이블로 존재하며, 인공적인 키(ID)를 통해 연결됩니다.

-  place_id, area_id와 같은 인공적 키로만 연결
-  데이터베이스는 ID가 무엇을 의미하는지 알지 못함
-  의미는 애플리케이션 로직에 의존

```
-- 인구 데이터 테이블
CREATE TABLE population (
  place_id INT,
  timestamp DATETIME,
  count INT,
  FOREIGN KEY (place_id) REFERENCES place(id)
);

-- 상권 데이터 테이블
CREATE TABLE commercial_area (
  area_id INT,
  place_id INT,
  business_count INT,
  FOREIGN KEY (place_id) REFERENCES place(id)
);
```

온톨로지 저장 구조

개념 중심 연결 구조

모든 정보가 :Place, :Road와 같은 의미 있는 개념을 중심으로 의미적으로 연결됩니다.

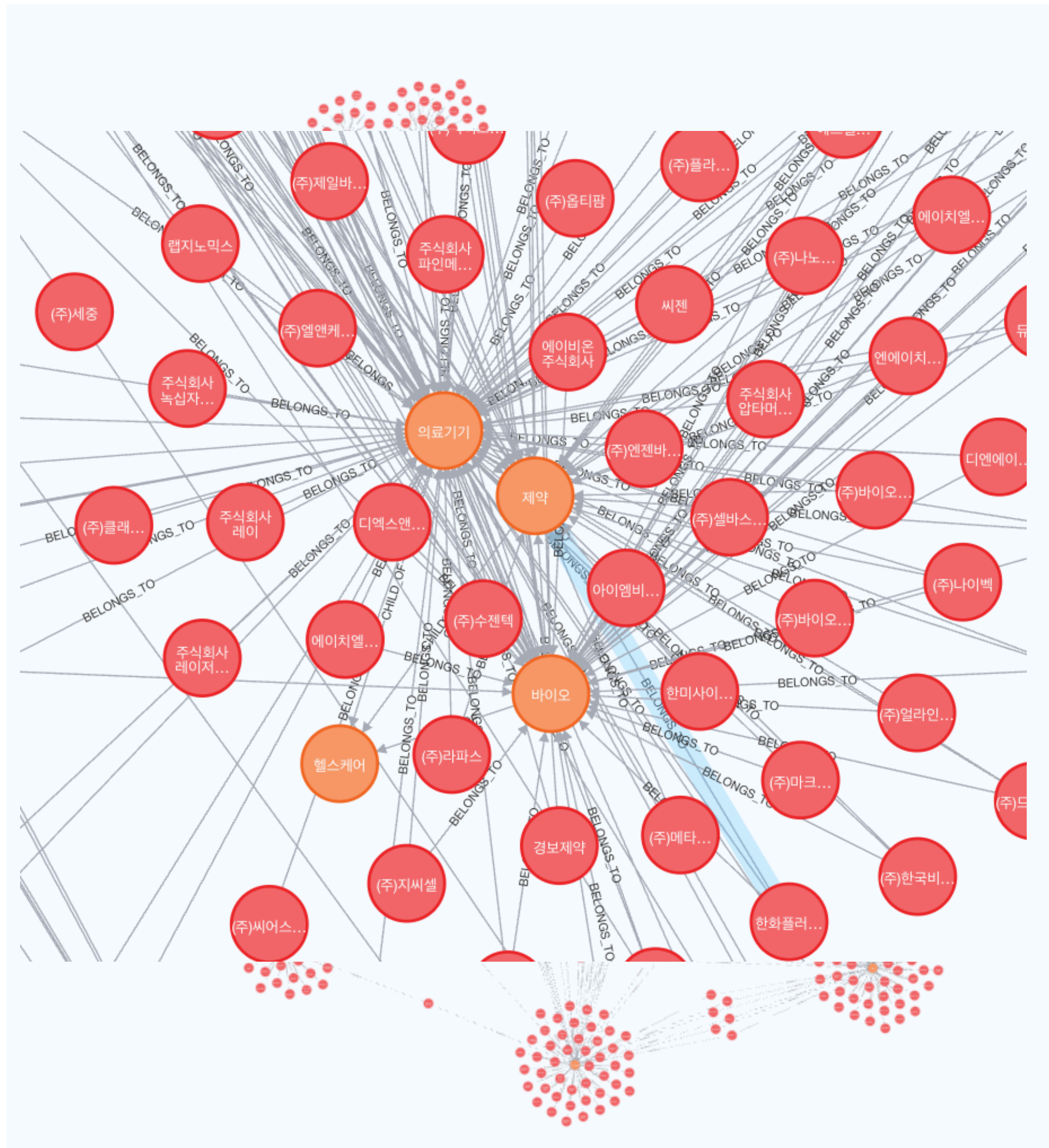
-  개념과 관계가 명시적 의미를 가진 URI로 표현
-  데이터 간의 의미적 연결이 데이터 자체에 내장
-  도시 지식망(Urban Knowledge Graph) 형성

RDF 트리플 예시:

```
:강남역 a :Place .
:강남역 :locatedIn :서울특별시 .
:강남역 :hasPopulation "45000"^^xsd:integer .
:강남역 :hasCommercialArea :강남상권 .
:강남상권 :hasBusinessCount "2300"^^xsd:integer .
```

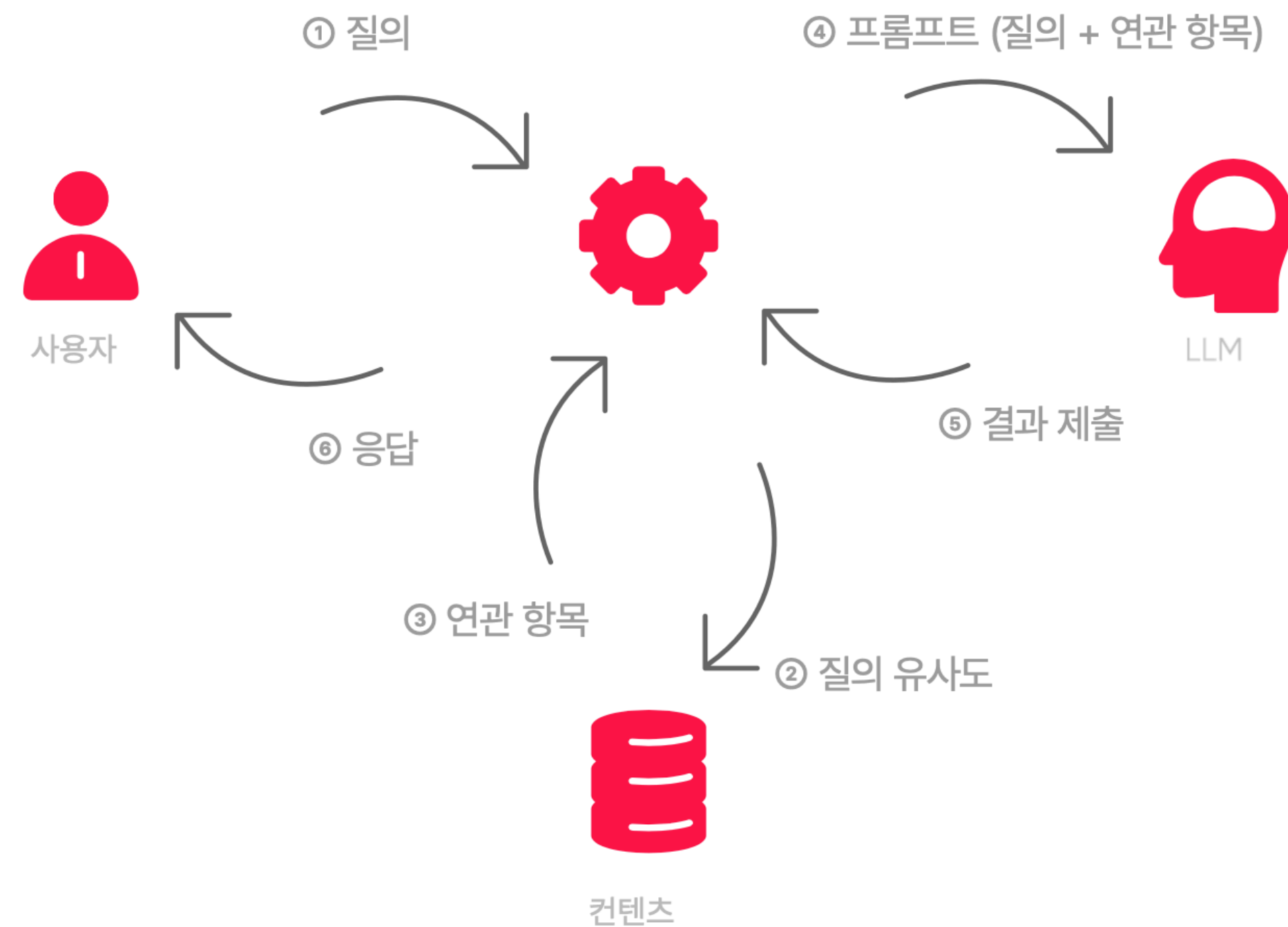
데이터가 지식으로

5. 지식 그래프



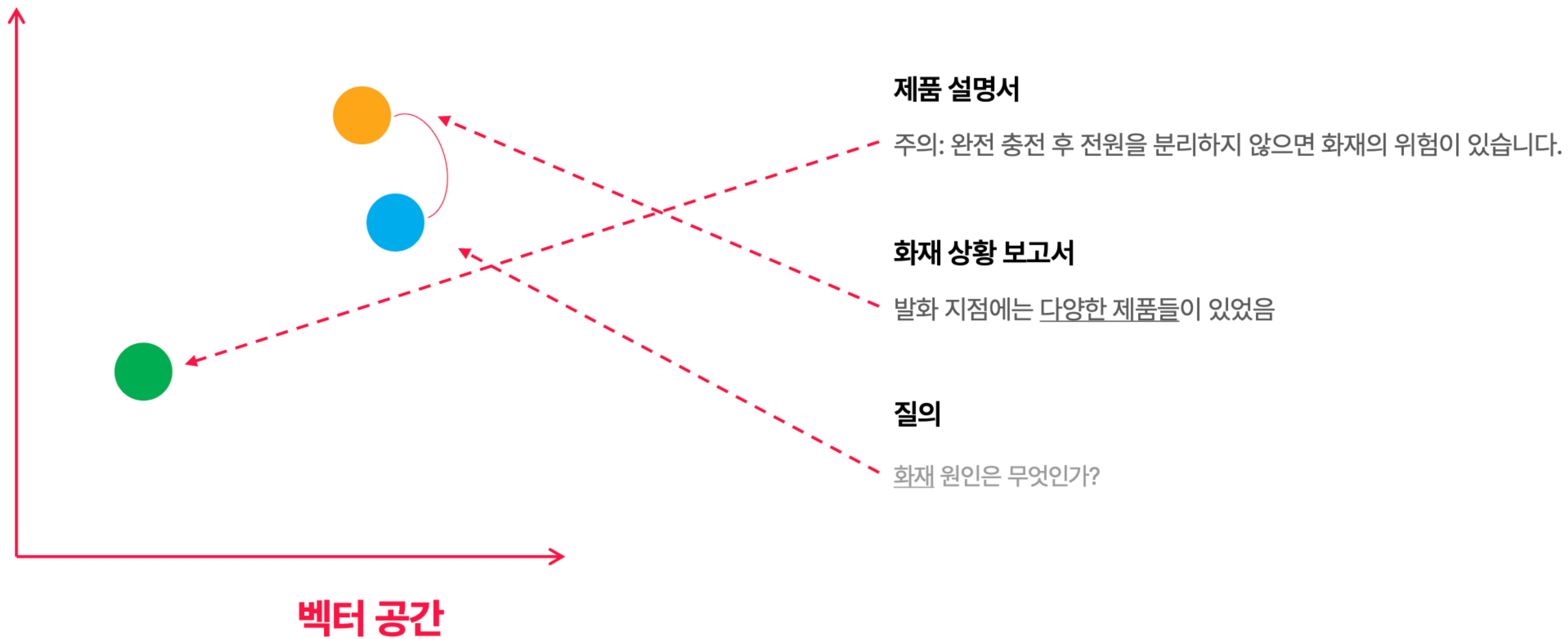
- Entity-Relation-Attribute 구조의 그래프형 지식베이스
- 식별 가능한 개체와 의미론적 관계를 연결
- 온톨로지 기반 시, 논리 제약과 추론 가능성 확보

6. RAG

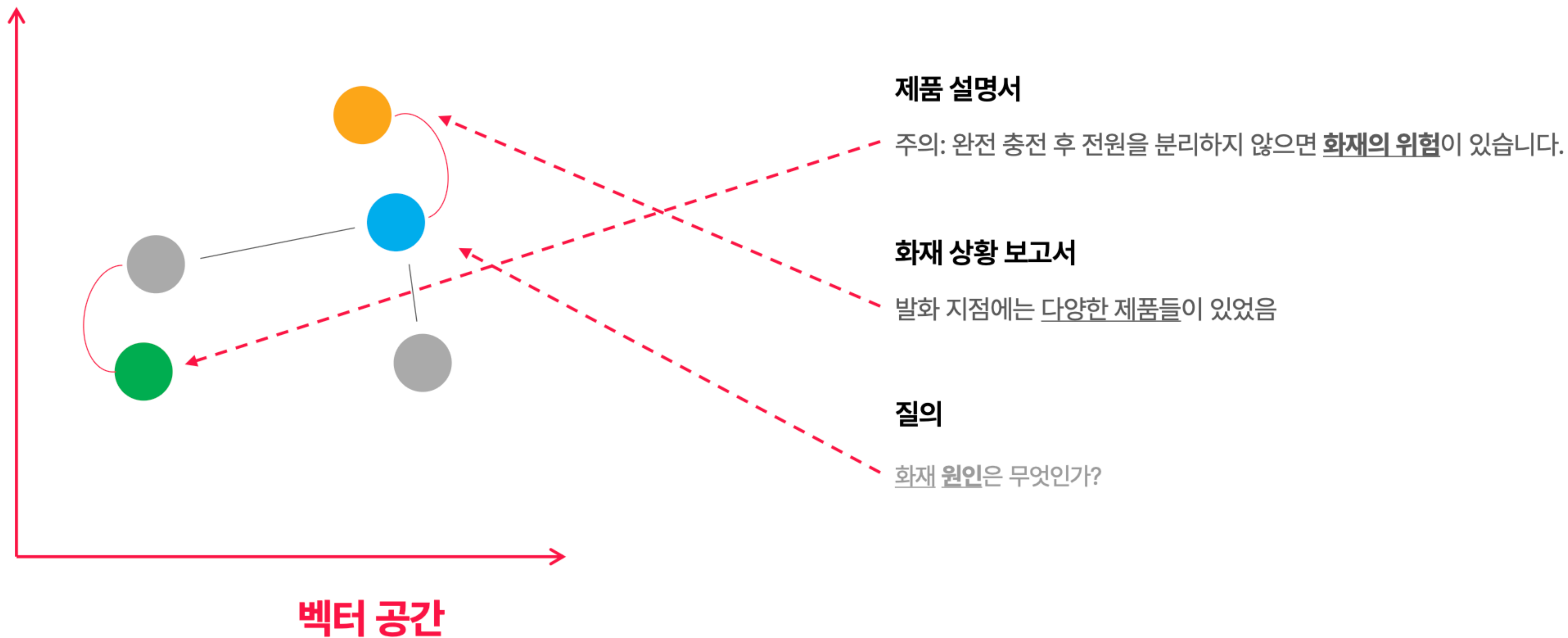


- **Retrieval(검색) : 텍스트 유사도 분석**
- **Augmented(증강) : 프롬프트 증강**
- **Generation(생성) : LLM 생성**

6-1. RAG



6-2. Graph RAG



6-3. RAG 와 GraphRAG 비교

- Standard RAG : 유사 문서 검색 + 생성
- GraphRAG : 그래프 기반 검색(개체/관계/제약) + 생성

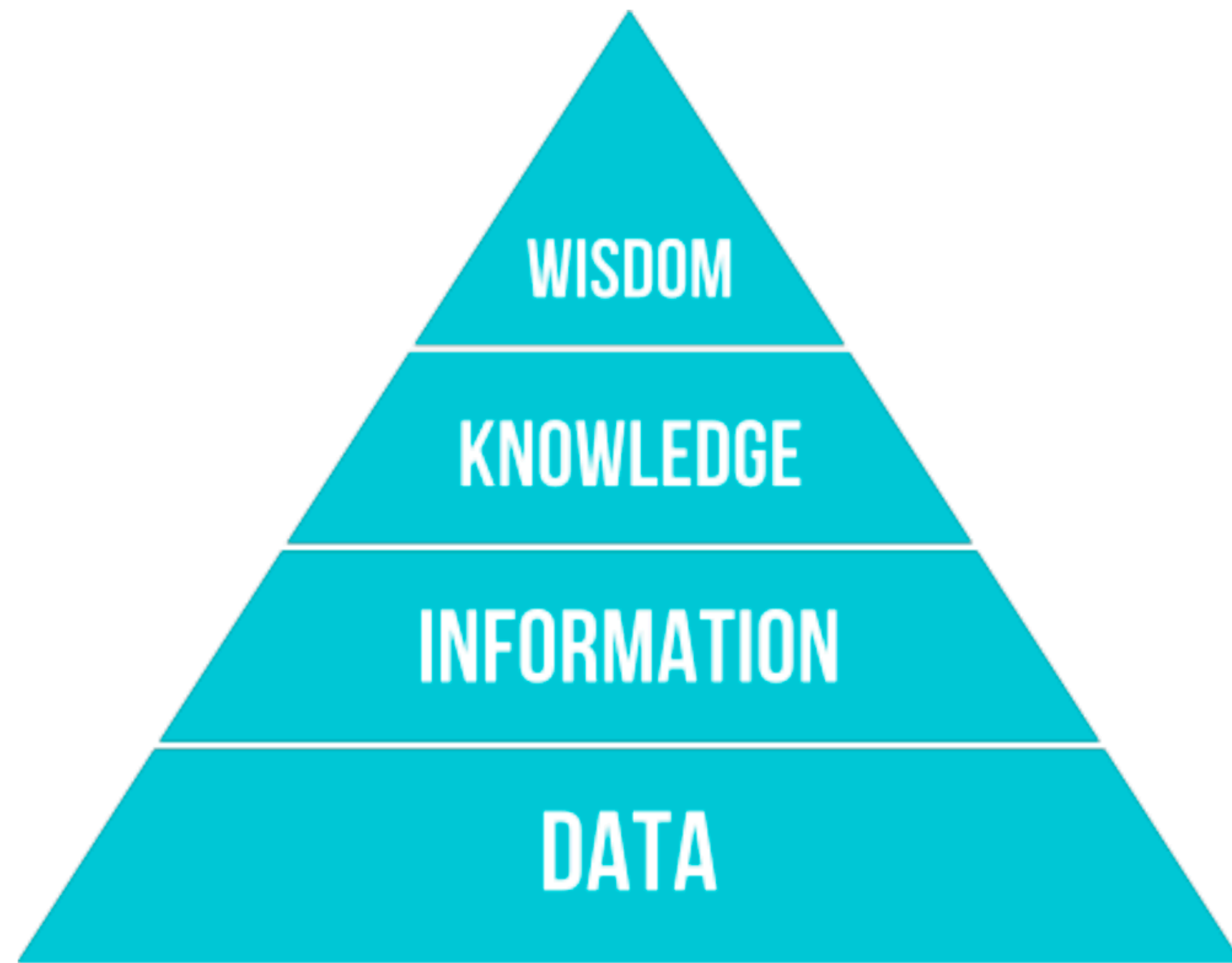
차이 :

Similarity retrieval -> Constraint-aware retrieval

비정형 근거 -> 구조화된 근거

설명 가능성 강화

8. Ontology AG (온톨로지 증강 생성)



Ontology AG

Ontology

정보의 연계

추론(inferencing)

Augmented

설명, 가설, 정책, 신뢰도

추리(reasoning)

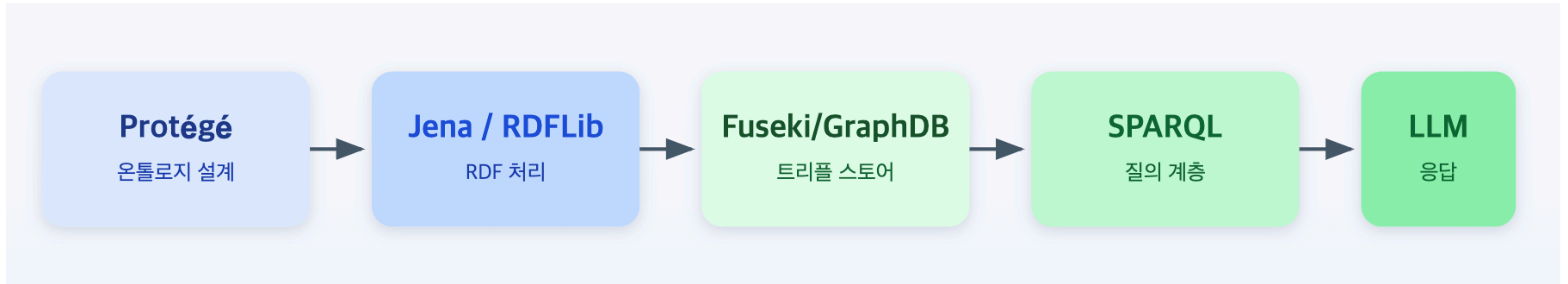
Generation

자연어 출력

LLM!

- 질문 -> 온톨로지 기반 해석/질의(SPARQL) -> 지식 그래프 조회 -> LLM 답변 생성
- RAG + 온톨로지/지식그래프

9. 도구 및 프레임워크



- Ontology editor : Protege
- RDF framework : Apache Jena, RDFLib
- Triple Store : Fuseki, GraphDB, Blazegraph
- Property Graph : Neo4j, Kuzu, Apache AGE